

Reviewer Pack — Minimal Order of Algebraic Topology Generators and SAT Claus...

Entry 87bd7c388ea1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 00:23:22 UTC

Minimal Order of Algebraic Topology Generators and SAT Clause Subset Complexity

Entry ID: 87bd7c388ea1 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 00:23:22 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Simplicial Complexes) **Field B** (complexity object): Boolean Satisfiability (SAT) Clause Subsets

Statement:

The minimal number of simplicial generators for the algebraic topology associated with a CNF formula is linearly correlated with its SAT clause subset complexity, such that the number of generators $N(G) = \Theta(\log^2(n))$, where n is the number of variables in the CNF.

Rationale (proposer's reasoning):

Algebraic topology provides a rich framework to study the structure of spaces and their properties. Simplicial complexes are fundamental objects in algebraic topology, and their generators can capture complex relationships within data structures. SAT clause subsets represent a combinatorial complexity inherent in solving SAT instances. This conjecture suggests that the simplicity of an associated simplicial complex is related to the complexity of clause subsets, potentially revealing new insights into the structure of SAT problems.

Taxonomy category: SIMP_LIM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a6042cf751e873e3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across 30 random seeds, the correlation coefficient between the minimal number of generators $N(G)$ and the SAT clause subset complexity exceeds 0.9 and $N(G)$ follows a distribution within $\pm 10\%$ of $\Theta(\log^2(n))$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - algebraic topology simplicial complexes AND boolean satisfiability clause subsets - minimal generators simplicial complexes CNF formula IN BOOLEAN MODE - CNF SAT clause subset complexity SIMPLEXES

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for i in range(n):
            clause = [random.randint(1, n), -random.randint(1, n)]
            while clause[0] == -clause[1]:
                clause[1] = -random.randint(1, n)
            clauses.append(clause)
        return clauses

    def simplicial_complex(cnf):
        # Simplified version for demonstration purposes
        return len(cnf) # This is a placeholder

    def sat_clause_subset_complexity(cnf):
        # Simplified version for demonstration purposes
        return sum(len(clause) for clause in cnf)

    n = random.randint(5, 40)
    cnf = generate_cnf(n)
    N_G = simplicial_complex(cnf)
    complexity = sat_clause_subset_complexity(cnf)

    if N_G <= 0 or complexity <= 0:
        return {
            "metric_name": "N(G)",
            "metric_value": N_G,
```

```

        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "Non-positive values"
    }

    metric_value = N_G
    instances_tested = 1
    n_max = n
    conjecture_holds = False
    counterexample = ""

    return {
        "metric_name": "N(G)",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 67, 71, 73, 79, 83, 89, 97]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = "N(G) does not follow  $O(\log^2(n))$ "
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

n_max': 18, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'N(G)', 'metric_value': 15, 'instances_tested': 1, 'n_max': 15, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'N(G)', 'metric_value': 7, 'instances_tested': 1, 'n_max': 7, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'N(G)', 'metric_value': 25, 'instances_tested': 1, 'n_max': 25, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'N(G)', 'metric_value': 23, 'instances_tested': 1, 'n_max': 23, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'N(G)', 'metric_value': 9, 'instances_tested': 1, 'n_max': 9, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'N(G)', 'metric_value': 38, 'instances_tested': 1, 'n_max': 38, 'conjecture_holds': False, 'counterexample': ''}

```

```

TRIAL: {'metric_name': 'N(G)', 'metric_value': 16, 'instances_tested': 1, 'n_max': 16, 'conjecture_ho
TRIAL: {'metric_name': 'N(G)', 'metric_value': 36, 'instances_tested': 1, 'n_max': 36, 'conjecture_ho
TRIAL: {'metric_name': 'N(G)', 'metric_value': 19, 'instances_tested': 1, 'n_max': 19, 'conjecture_ho
TRIAL: {'metric_name': 'N(G)', 'metric_value': 17, 'instances_tested': 1, 'n_max': 17, 'conjecture_ho

```

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_78bbfd87.py", line 86, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_78bbfd87.py", line 86, in <genexpr>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~^

```

KeyError: 'seed'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents a proper evaluation of the conjecture. | next: Review and debug the test code to ensure it can complete without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	20207
2	preregistration	ollama_remote	glm4:latest	0	0	9487
3	novelty	ollama_remote	glm4:latest	0	0	8677
4	novelty	ollama_remote	glm4:latest	0	0	11571
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11773
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14917
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8559
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9222
9	judge	ollama_remote	glm4:latest	0	0	11947

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 106360 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/87bd7c388ea1.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/87bd7c388ea1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/87bd7c388ea1.tar.gz` (if generated)